

ui.textarea 全面详细阐述

`ui.textarea` 是 NiceGUI 框架中基于 Quasar QInput 组件实现的多行文本输入组件，默认启用多行输入模式，支持文本实时监听、表单验证、一键清除、组件绑定等核心功能，适用于备注填写、评论提交、内容编辑等需要输入大量文本的场景。以下从核心特性、使用方法、参数配置、API 详情等方面展开说明。

一、核心特性

- 多行输入支持**：默认呈现多行文本框形态，自动适配文本换行，无需额外配置即可输入长文本。
- 与输入框同源能力**：继承 `ui.input` 的核心功能，包括标签显示、占位提示、实时值监听、灵活验证机制（字典规则 / 自定义函数 / 异步验证）。
- 便捷操作功能**：支持 Quasar 的 `clearable` 属性，添加一键清除按钮，快速清空输入内容。
- 高度可扩展性**：支持样式自定义（通过 `classes / style / props`）、组件状态绑定（启用 / 禁用、显示 / 隐藏、标签 / 值绑定），适配各类界面需求。
- 版本兼容特性**：2.7.0+ 版本支持异步验证函数，2.16.0+ 支持 `html_id` 属性，3.0.0+ 支持绑定参数的 `strict` 校验。

二、基础使用方法

1. 最简示例：实时监听输入

通过 `on_change` 回调实时响应文本变化，同步显示输入内容：

```
from nicegui import ui

# 标签用于展示输入结果
result = ui.label()

# 基础多行文本输入框
ui.textarea(
    label='多行文本输入',
    placeholder='开始输入...（支持换行）',
    # 实时监听输入变化，更新标签内容
    on_change=lambda e: result.set_text(f'你输入的内容: \n{e.value}')
)

ui.run()
```

2. 一键清除功能

通过 `props('clearable')` 启用内置清除按钮，点击即可清空输入内容：

```

from nicegui import ui

# 启用清除功能的文本框
textarea = ui.textarea(
    value='默认文本内容（可一键清除）',
    label='可清除文本框'
).props('clearable') # 启用清除按钮

# 标签与文本框值绑定，实时同步显示
ui.label().bind_text_from(textarea, 'value')

ui.run()

```

3. 输入验证（两种方式）

支持字典式规则验证和自定义函数验证，支持异步验证（2.7.0+ 版本）：

```

from nicegui import ui
import asyncio

# 方式1: 字典规则验证（键为错误提示，值为验证函数）
ui.textarea(
    label='文本长度验证（字典式）',
    placeholder='输入内容需少于10个字符',
    validation={'输入过长!': lambda value: len(value) < 10}
)

# 方式2: 自定义函数验证（返回错误提示或 None）
ui.textarea(
    label='文本长度验证（函数式）',
    placeholder='输入内容需不少于5个字符',
    validation=lambda value: '输入过短!' if len(value) < 5 else None
)

# 方式3: 异步验证（2.7.0+ 版本）
async def async_validate(value):
    await asyncio.sleep(0.8) # 模拟异步操作（如后端接口校验）
    return '包含敏感词' if '敏感词' in value else None

ui.textarea(
    label='异步验证（含敏感词检测）',
    placeholder='输入内容...',
    validation=async_validate
)

ui.run()

```

4. 禁用自动验证

通过 `without_auto_validation()` 方法关闭实时自动验证，仅手动触发验证：

```
from nicegui import ui

# 禁用自动验证，仅支持手动触发
textarea = ui.textarea(
    label='手动触发验证',
    placeholder='输入内容需介于3-20个字符',
    validation=lambda value:
        '过短（最少3个字符）' if len(value) < 3
        else '过长（最多20个字符）' if len(value) > 20
        else None
).without_auto_validation()

# 验证按钮：点击触发验证并显示结果
def trigger_validate():
    is_valid = textarea.validate() # 手动触发验证
    error_msg = textarea.error or '验证通过'
    ui.notify(f'验证结果: {error_msg}')

ui.button('点击验证', on_click=trigger_validate)

ui.run()
```

三、关键参数说明

- `label`：类型为 `str | None`，文本框的显示标签，用于说明输入用途，无默认值。
- `placeholder`：类型为 `str | None`，文本框为空时显示的提示文本，无默认值。
- `value`：类型为 `str | None`，文本框的初始值，默认值为 `None`。
- `on_change`：类型为 `Callable[[ValueChangeEventArguments], Any] | Callable[[], Any]`，输入值变化时（每一次按键或换行）触发的回调函数，事件对象 `e` 包含 `value` 属性（当前输入值），无默认值。
- `validation`：类型为 `dict[str, Callable[[Any], bool]] | Callable[[Any], str | None | Awaitable[Optional[str]]] | None`，输入验证规则。支持字典（错误提示→验证函数）或自定义函数（返回错误提示 / `None`，支持异步），默认值为 `None`（无验证）。

四、高级功能

1. 组件双向绑定

将文本框与其他组件（如标签、开关）绑定，实现值或状态同步：

```

from nicegui import ui

# 文本框与标签双向绑定（值同步）
textarea = ui.textarea(label='双向绑定示例', value='初始内容')
label = ui.label('当前内容: 初始内容').bind_text_from(textarea, 'value', forward=lambda x: f'当前内容: {x}')

# 开关控制文本框启用/禁用
enable_switch = ui.switch(label='启用文本框', value=True)
enable_switch.bind_value_to(textarea, 'enabled')

ui.run()

```

2. 动态修改属性（标签、值）

通过 `set_label()`、`set_value()` 等方法动态更新文本框属性：

```

from nicegui import ui

textarea = ui.textarea(label='初始标签', placeholder='输入内容...')

# 动态修改标签
ui.button('修改标签', on_click=lambda: textarea.set_label('动态更新的标签'))

# 动态设置默认值
ui.button('填充默认内容', on_click=lambda: textarea.set_value('这是动态填充的默认多行文本\n支持换行显示'))

ui.run()

```

3. 样式自定义

通过 `props`、`classes`、`style` 自定义文本框外观，适配界面风格：

```

from nicegui import ui

# 1. 使用 Quasar 原生属性（圆角、边框、紧凑模式）
ui.textarea(
    placeholder='圆角+边框+紧凑模式',
    props='rounded outlined dense'
)

# 2. 自定义样式（宽度、高度、字体、内边距）
ui.textarea(
    label='自定义样式',
    value='蓝色字体、固定高度、灰色背景',
    classes='w-96 bg-slate-50', # Tailwind 类: 宽度、背景色
    style='height: 120px; color: blue; padding: 10px;' # 内联 CSS: 高度、字体颜色、内边距
)

ui.run()

```

4. 延迟确认输入（回车 / 失去焦点）

若无需实时监听，通过 `keydown.enter`（回车）或 `blur`（失去焦点）事件触发延迟回调：

```
from nicegui import ui

result = ui.label('确认后显示内容...')

textarea = ui.textarea(
    label='延迟确认输入',
    placeholder='按回车或失去焦点确认'
)

# 回车触发确认
textarea.on('keydown.enter', lambda e: result.set_text(f'回车确认: \n{e.sender.value}'))

# 失去焦点触发确认
textarea.on('blur', lambda e: result.set_text(f'失去焦点确认: \n{e.sender.value}'))

ui.run()
```

五、API 详情补充

1. 核心属性

属性名	类型	说明		
<code>classes</code>	<code>str</code>	组件的 HTML 类名，支持 Tailwind/Quasar 类，用于容器样式自定义		
<code>client</code>	<code>Client</code>	组件所属的客户端实例		
<code>enabled</code>	<code>BindableProperty</code>	是否启用组件（可绑定，支持动态切换）		
<code>error</code>	<code>`str</code>	<code>None`</code>	验证失败后的错误信息（可设置，实时反馈验证结果）	
<code>html_id</code>	<code>str</code>	组件的 HTML DOM ID（版本 2.16.0+），用于手动定位 DOM 元素		
<code>label</code>	<code>BindableProperty</code>	文本框标签（可绑定，支持动态修改）		
<code>props</code>	<code>Props[Self]</code>	组件的 Quasar 原生属性（如 <code>clearable</code> 、 <code>rounded</code> 、 <code>outlined</code> ）		

属性名	类型	说明		
<code>style</code>	<code>style[self]</code>	组件容器的内联 CSS 样式 (如宽度、高度、颜色)		
<code>validation</code>	<code>`Callable`</code>	<code>dict</code>	<code>None`</code>	验证规则 (可设置, 支持动态修改)
<code>value</code>	<code>BindableProperty</code>	文本框当前值 (可绑定, 支持动态同步)		
<code>visible</code>	<code>BindableProperty</code>	组件是否可见 (可绑定, 支持动态切换)		

2. 常用方法

方法名	说明	参数
<code>bind_enabled(target)</code>	双向绑定组件启用状态到目标对象	<code>target</code> : 目标组件; <code>target_name</code> : 目标属性名 (默认 <code>enabled</code>)
<code>bind_label(target)</code>	双向绑定组件标签到目标对象	<code>target</code> : 目标组件; <code>target_name</code> : 目标属性名 (默认 <code>label</code>)
<code>bind_value(target)</code>	双向绑定组件值到目标对象	<code>target</code> : 目标组件; <code>target_name</code> : 目标属性名 (默认 <code>value</code>)
<code>bind_visibility(target)</code>	双向绑定组件可见性到目标对象	<code>target</code> : 目标组件; <code>target_name</code> : 目标属性名 (默认 <code>visible</code>)
<code>set_enabled(bool)</code>	启用 / 禁用组件	<code>bool</code> : <code>True</code> 启用, <code>False</code> 禁用
<code>set_label(str)</code>	动态修改文本框标签	<code>str</code> : 新标签 (<code>None</code> 隐藏标签)
<code>set_value(str)</code>	动态设置文本框值	<code>str</code> : 新输入值 (支持换行符 <code>\n</code>)
<code>set_visibility(bool)</code>	显示 / 隐藏组件	<code>bool</code> : <code>True</code> 显示, <code>False</code> 隐藏
<code>validate(return_result=True)</code>	手动触发验证	<code>return_result</code> : 是否返回验证结果 (异步验证需设为 <code>False</code>) ; 返回值: <code>True</code> 有效 / 错误信息
<code>without_auto_validation()</code>	禁用实时自动验证	无参数, 返回组件实例 (支持链式调用)

方法名	说明	参数
<code>tooltip(text)</code>	为组件添加悬浮提示	<code>text</code> : 提示文本内容
<code>update()</code>	强制更新组件状态到客户端	无参数

3. 事件说明

- `on_change`: 文本值实时变化（每一次按键、换行），事件对象含 `value` 属性。
- `keydown.enter`: 按下回车键触发，适用于手动确认输入内容。
- `blur`: 文本框失去焦点时触发，适用于离开输入区域后确认内容。
- `click`: 点击文本框时触发，可用于初始化操作（如加载默认文本）。

六、注意事项

1. **多行特性差异**: 与 `ui.input` 相比，`ui.textarea` 默认多行，无需额外配置换行，输入时按 `Enter` 键直接换行，而非触发确认（需通过 `keydown.enter` 事件手动实现确认逻辑）。
2. **验证功能使用**: 异步验证函数需返回 `Awaitable[optional[str]]`，调用 `validate()` 时需设置 `return_result=False`，验证结果在后台异步处理。
3. **样式自定义限制**: 底层基于 Quasar QInput，需通过 `props`（如 `input-style`/`input-class`）样式化内部原生输入框，而非直接样式化外层容器。
4. **值绑定换行处理**: 绑定目标组件（如标签）时，需确保目标支持换行显示（如标签默认支持 `\n` 换行，无需额外处理）。
5. **版本兼容性**: 使用 `html_id` 需 2.16.0+ 版本，异步验证需 2.7.0+ 版本，`strict` 参数绑定需 3.0.0+ 版本，需根据实际框架版本调整功能使用。

七、应用场景

1. 备注 / 说明填写: 表单中收集用户详细备注、补充说明等长文本信息。
2. 评论 / 反馈提交: 允许用户输入多行评论、问题反馈，支持实时验证内容长度。
3. 内容编辑: 简单的文本编辑场景（如编辑短文、修改描述），支持一键清除重写。
4. 异步校验场景: 如内容合规检测、敏感词过滤，通过异步验证函数对接后端接口。
5. 动态交互面板: 与开关、按钮等组件绑定，实现文本框状态（启用 / 禁用、显示 / 隐藏）的动态控制。

`ui.textarea` 继承了 `ui.input` 的便捷性与扩展性，同时专注于多行文本输入场景，通过简洁的 API 满足长文本输入的各类需求，是 NiceGUI 中处理大量文本输入的核心组件。